Implementation of RL-based Intelligent Control System on a Gyroscope Setup

S. Sundar Palaniappan[1]
Dept. of Instrumentation and Control Engg., NIT Tiruchirappalli
Tiruchirappalli, India
110123109@nitt.edu

Mohan Aneesh Sai Pusarla[2]
Dept. of Mechanical Engg., NIT Tiruchirappalli
Tiruchirappalli, India
111123083@nitt.edu

Imran Sayyed[3]
Dept. of Aerospace Engg., IIT Madras
Chennai, India
imransayyed23@gmail.com

Nandan K. Sinha[4]
Dept. of Aerospace Engg., IIT Madras
Chennai, India
nandan@ae.iitm.ac.in

***Abstract*— This paper presents the simulation and hardware implementation of a deep reinforcement learning(DRL) control strategy for reference tracking in a three degree of freedom(3-DOF) Quanser gyroscope. The objective is to develop a controller using the DRL algorithm Proximal Policy Optimization(PPO) to achieve precise angular control. The dynamic model is formulated based on Euler-Lagrange's equations and expressed in state-space representation to represent the system. The gyroscope environment is modeled in Simulink with a custom made reward function and observations vector. The algorithm works based on actor-critic architecture and is developed in MATLAB using Reinforcement Learning Toolbox. The PPO agent is trained for random inputs to minimize tracking error. The best trained agent is evaluated in both Simulink and gyroscope hardware to assess it's performance.**

I. INTRODUCTION

A gyroscope is a device comprising a spinning wheel or disk mounted within a system of gimbals, allowing it to maintain its orientation independently of its mounting structure. The foundational principles of this instrument were first documented by Johann Bohnenberger in 1817. Since then, gyroscopes have become indispensable in diverse fields, ranging from sophisticated aerospace navigation and control systems in aircraft, spacecraft, and missiles, to everyday portable electronic devices. In aerospace, gyroscopes are critical for determining and maintaining a vehicle's orientation (attitude) and angular velocity by leveraging the principle of angular momentum, thus providing a stable reference frame.

A primary control objective is reference tracking, which involves achieving a specified angular orientation from an initial state. For systems, particularly those that are linearized, conventional control strategies such as Proportional-Integral-Derivative (PID) controllers and Linear-Quadratic Regulators (LQR) are frequently employed. However, a significant drawback of these methods lies is due to the extensive and iterative tuning of gain parameters to achieve desired performance and stability, which can be a time-consuming and complex process.

In recent years, there has been growing interest in the application of machine learning techniques to control systems, particularly in the use of reinforcement learning (RL) for decision-making and control tasks. RL is a reward-based learning framework in which an agent learns to make optimal decisions through interaction with an environment, receiving feedback in the form of scalar rewards.To address this, deep-reinforcement learning (DRL) methods have been developed, using deep neural networks to approximate policies and value functions in continuous domains. Algorithms such as the deep deterministic policy gradient (DDPG), proximal policy optimization (PPO), twin delayed deep deterministic policy gradient (TD3), trust region policy optimization (TRPO) and soft actor-critic (SAC) have shown significant results in continuous control tasks.

In this work, Proximal Policy Gradient(PPO) algorithm is implemented in a Simulink-based simulation of a 3-Degree Of Freedom (3-DOF) gyroscope, with the RL implementation coded in MATLAB. The performance is evaluated by training the model with randomized inputs. The PPO model was implemented This work serves useful for future efforts in applying DRL to real-world control problems, including nonlinear or uncertain systems.

[1] and [2] were B.Tech second year summer interns at IIT Madras from NIT Tiruchirappalli.
[3] is a MS Research Scholar at Aerospace Deparatment at IIT Madras.
[4] is a Professor in Flight Dynamics Lab, Aerospace Department at IIT Madras.

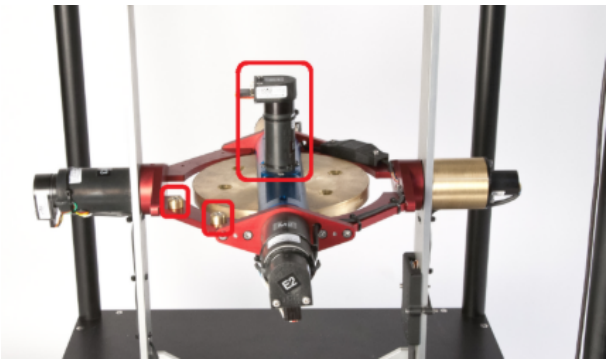


Fig. 1. Gyroscope Setup

II. SYSTEM DESCRIPTION AND MODELING

A. Gyroscope Setup

The experimental setup consists of a three-degrees-of-freedom (3-DOF) gyroscope system, as shown in Fig. 1. The mechanical assembly comprises a central disk mounted within an inner (blue) gimbal, which is further nested inside an outer (red) gimbal. This gimbal structure is supported by a rectangular frame capable of rotation about its vertical axis. For the purposes of this study, the frame is locked in place.

For actuation, the system integrates four DC motors: one dedicated to the disk's spin axis, two for independently driving the blue and red gimbals, and a fourth motor for controlling the gray rectangular frame, which serves as a mechanism for introducing controlled disturbances. Angular positions for all axes are measured via optical encoders.

B. Equations of Motion

The equations of motion are very important to represent the gyroscope system in the developed model. These are derived using the Euler-Lagrange's equations and by defining the frames of the gyroscope. The relative velocity among the frames is used to find the Kinetic Energy of the system.

$$L(q,\dot{q}) = T(q,\dot{q}) - V(q,\dot{q}) \tag{1}$$

where L is the Lagrange, T is the Kinetic Energy and V is the potential energy. In this paper, V(q,$\dot{q}$) = 0. The Euler-Lagrange's equation is

$$\frac{d}{dt}\left(\frac{\partial L(q,\dot{q})}{\partial \dot{q}_i}\right) - \frac{\partial L(q,\dot{q})}{\partial q_i} = \tau_i, \quad i = 1,\ldots,n_q \tag{2}$$

where q and $\dot{q}$ are the relative angles and angular velocities, respectively. τ_i is the input torque given to frame of number i. The formulation follows the modeling approach as presented in [1]

Using these equations as basis, as outlined in [2], the equations of motion of the gyroscope are ,

$$\begin{aligned} J_y\ddot{\theta} + h\dot{\psi} &= \tau_y \\ J_z\ddot{\psi} - h\dot{\theta} &= 0 \end{aligned} \tag{3}$$

where,

$$J_y = 0.0039\ Kg - m^2$$

$$J_z = 0.0223\ Kg - m^2$$

$$h = 0.044\ Kg - m^2/s$$

C. State-Space Model

The state-space equations are

$$\dot{x} = Ax + Bu \tag{4}$$

$$y = Cx + Du \tag{5}$$

$$x = \begin{bmatrix} \dot{\theta} & \psi & \dot{\psi} & \int \psi \end{bmatrix}^\top \tag{6}$$

$$y = \psi \tag{7}$$

where:

- $x \in \mathbb{R}^4$ is the state vector,
- $u \in \mathbb{R}^1$ is the control input,
- $y \in \mathbb{R}^1$ is the output,
- $A \in \mathbb{R}^{4\times4}$, $B \in \mathbb{R}^{4\times1}$, $C \in \mathbb{R}^{4\times4}$, and $D \in \mathbb{R}^{1\times4}$ are system matrices.

The states are :-

- $\dot{\theta}$ is the angular velocity of blue gimbal,
- ψ is the angle of red gimbal,
- $\dot{\psi}$ is the angular velocity of red gimbal,
- $\int \psi\, dt$ is the integral of ψ

In this system, the torque τ is the control input given to the red gimbal, which changes the angle θ. This controls the angle ψ which is the control variable.

The numerical values of A,B,C,D are :-

$$A = \begin{bmatrix} 0 & 0 & -\frac{h}{J_y} & 0 \\ 0 & 0 & 1 & 0 \\ \frac{h}{J_z} & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & J_y & 0 & 0 & 0 \end{bmatrix}^\top$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$D = \begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix}$$

The values of system matrices used for modeling the gyroscope are obtained from [2]

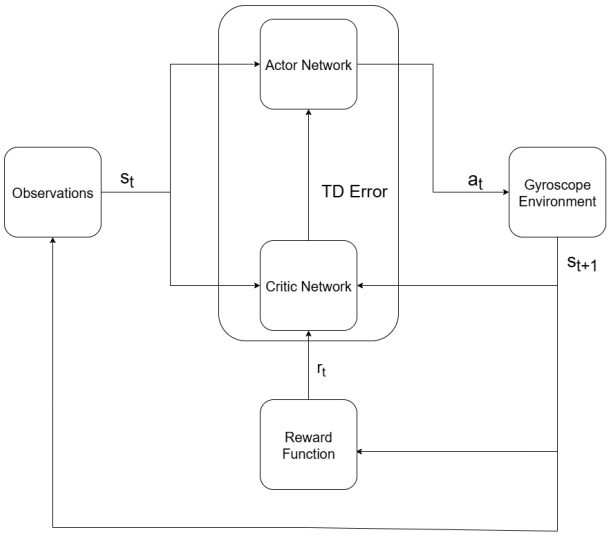


Fig. 2. Actor-Critic Framework of PPO model

III. Methodology - Deep Reinforcement Learning for Continuous Control

Many dynamic systems such as the gyroscope, operate in environments where both the observed states and action space are continuous. Designing controllers for such systems requires algorithms that can handle these continuous spaces effectively. Policy Gradient methods are well-suited to the task, as they optimize a parameterized policy directly using a differentiable loss function and gradient-based techniques like backpropagation to tune the neural networks. Unlike traditional value-based methods that adjust the policy indirectly, Policy Gradient approaches update the policy itself in response to performance. Modern versions often use an Actor-Critic architecture, which allows the system to learn and adapt efficiently in continuous control settings.

A. Actor Critic Architecture

The Actor-Critic architecture as shown in Fig.2, is a widely used framework for modern reinforcement learning algorithms, particularly for Policy Gradient methods. The actor is a neural network parameterized by θ, representing a policy $\pi_\theta(a|s)$, which maps the current state s to a probability distribution over actions (for stochastic policies like PPO). The critic is a separate neural network that estimates a value function $V^\pi(s)$ to assess how good the action is. During training, the current state s is observed, and the actor performs an action a_t, which leads to the next state s_{t+1} and a reward r_t. The state s_{t+1} is used to compute a TD error between the current value function $V(s_t)$ and the target value $r_t + \gamma V(s_{t+1})$ (s_{t+1} is fed to the critic network without updating the weights) and this is used to update the weights of the critic. The actor uses this TD error as feedback and uses gradient descent to update the weights and improve the policy.

The MSE loss is

$$\mathcal{L}_{\text{critic}} = \left(r_t + \gamma V(s_{t+1}) - V(s_t)\right)^2 \tag{8}$$

This is used to update the weights of the critic by backpropagation.

$$\phi \leftarrow \phi - \alpha \nabla_\phi L_{\text{critic}}(\phi) \tag{9}$$

The actor maximizes the expected return by using the policy gradient :

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot \delta_t\right] \tag{10}$$

where $J(\theta)$ is the objective function that is maximized using gradient ascent.

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta) \tag{11}$$

B. Proximal Policy Gradient(PPO):-

Proximal Policy Optimization (PPO), introduced by Schulman et al. [3], is an on-policy reinforcement learning algorithm widely used in continuous control tasks for its training stability and sample efficiency. Its advantage lies in the clipped surrogate objective, which clips large policy updates that can destabilize learning. This makes PPO more stable than other policy gradient methods, which often cause oscillatory behaviors. While it may require more training episodes, PPO typically produces smoother and more reliable control outputs, making it well-suited for precision tasks.

The surrogate loss function used by PPO is :

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\,\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\right)\right] \tag{12}$$

where:

- $r_t(\theta) = \dfrac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$ is the probability ratio between the current and the old policy.
- $\hat{A}_t$ denotes the advantage estimate at timestep t, which measures how much better an action is compared to the average.
- ϵ is a small positive hyperparameter that constrains the policy update, helping to ensure stable and conservative improvements.

The clipped loss function $L^{\text{CLIP}}(\theta)$ encourages improvement when $\hat{A} > 0$ and prevents large updates by clipping $r_t(\theta)$ if it deviates too much. The actor's network weights are then updated using the gradient ascent on this clipped loss function.

$$\theta \leftarrow \theta + \alpha \nabla_\theta L_{\text{CLIP}}(\theta) \tag{13}$$

The weights of critic are updated as shown in (9).

IV. Agent Design

In reinforcement learning, an agent observes the environment and selects actions based on its policy, aiming to maximize rewards. The environment responds with a new state and reward, which is used for learning. A well-designed reward structure is developed to penalize undesired behavior and encourage goal-directed actions.

A. Observations for agent

In this work, the agent's observations is composed of selected set of states from the simulink model that are important for the system's dynamics. These variables are used by the actor network to output an action. The agent receives the following states in the form of observation vector:

$$\begin{bmatrix} e & \int e\,dt & \dot{\theta} & \psi & \dot{\psi} & \int \psi\,dt \end{bmatrix}^\top \tag{14}$$

where e is the error between target and control variable ψ. The observations are selected to represent the dynamics of the system. The states $\dot{\theta}$,ψ,$\dot{\psi}$,$\int \psi\,dt$ were used to derive the state-space equation matrices and model the environment. The error term is computed by taking the difference between reference and feedback. ψ is taken as feedback and is compared with the target to minimize error by feeding this error to the actor. Additionally, the integral term $\int e\,dt$ is used as an observation as it represents all accumulated errors over time. Integral terms such as $\int e\,dt$ and $\int \psi\,dt$ are used to reduce oscillations and eliminate steady-state errors. These are essential in systems such as gyroscopes where damping is limited and oscillations can persist.

B. Reward Structure

The reward function is:

$$r = -e^2 + \mathbf{1}_{|e|\leq 0.01} - 0.1\cdot\mathbf{1}_{|e|>0.01} - 100\cdot\mathbf{1}_{\psi\notin(-\pi,\pi)} - 0.1\int \psi dt \tag{15}$$

The reward structure is designed as a continuous function to encourage smooth learning.The term $-e^2$ ensures that larger errors are penalized more heavily. A positive reward of +1 is given for actions which yield an error below 0.01 radians (0.57 degrees) to encourage the agent to improve its actions and ensure that the error is below this limit. A negative reward (penalty) of -0.1 is assigned to actions that lead to higher errors, thereby discouraging the agent from repeating such actions in the future. This negative reward is kept significantly lesser than the positive reward to avoid sparse rewards which cause delayed convergence and slow learning. A large penalty of -100 is given for actions which cause the control variable ψ to exceed its allowable range. This acts as an hard constraint which penalizes the agent heavily for doing actions that drive ψ in the region $(-\infty,-\pi)\cup(\pi,\infty)$. The integral term $\int \psi dt$ improves the control response by reducing the steady-state error and oscillations.

V. RL PPO algorithm implementation

The reinforcement learning algorithms are implemented in MATLAB using the Reinforcement Learning Toolbox [4]. The agent receives continuous 6-dimension vector for observations that are unbounded except e and ψ which ranges from $[-\pi,\pi]$. The action output by the agent is torque, that actuates the blue gimbal to change the control variable ψ. In this study, the targets and initial conditions are randomized for training. A local reset function is used to reset the environment during training after one episode to reinitialize target and initial conditions.

A. Critic Network

The critic network estimates the value function at the current state based on the input given as observation vector. The network has 3 fully connected layers with 100 neurons and utilize ReLU activations.

B. Actor Network

The actor network has 3 fully connected layers with 100 neurons in its common path. The mean path and standard deviation paths have 50 neurons in its input layer. The actor network outputs a stochastic policy by modeling the action as a gaussian probability distribution. The mean(μ and standard deviation(σ of the distribution is used for sampling the action. The action is sampled as :

$$a \sim \mathcal{N}(\mu(s), \sigma(s)^2) \tag{16}$$

The actor network consists of three branches: a shared input path, a mean path, and a standard deviation path. The input path processes the observation vector through three fully connected layers (100 neurons each) with ReLU activations, producing a shared feature. This feature is passed to both the mean and standard deviation branches. In the mean path, it flows through a 50-neuron ReLU layer and a single output neuron (scalar action), followed by a Tanh layer to constrain the output to (-1,1) which is necessary for bi-directional actions like torque, and then a scaling layer to match the required torque range. The standard deviation path has a similar structure as mean path, but it ends with a SoftPlus layer to ensure positive outputs. The final action is sampled from the resulting Gaussian distribution.

C. Hyper Parameters

The PPO agent's performance during training depends on few hyper-parameters that can be set. These hyper-parameters are chosen based on standard practices.

- Experience Horizon = 1024

- Discount Factor (γ) = 0.95
- Critic learning rate = 3 x 10^{-4}
- Actor learning rate = 5 x 10^{-4}
- Gradient Threshold for both actor and critic = 1
- Entropy Loss Weight = 0.001
- Clipping Factor (ϵ) = 0.4

VI. Training Setup and Evaluation

The reinforcement learning agent was trained using MATLAB's reinforcement learning toolbox and the training progress was monitored in the built-in RL Training monitor which provided visual feedback on reward trends, episode durations, and agent performance.

In the training setup, the target reference was also randomized. The target value was randomly selected from a discrete set of reference angles $[10°, 15°, 23°, 30°, 37°, 43°, 54°, 62°, 70°, 75°]$. This adds variability in both references and initial condition to train the agent to develop a robust policy. The agent with the highest reward is chosen for validation.

VII. Simulation of Best Trained Agent

Following the training phase of the reinforcement learning agent, the most effective policy, identified by achieving the maximum cumulative reward, was selected for validation. This selected agent was then evaluated by exposing it to a variety of input signals within the simulation environment. The agent's performance and operational characteristics were subsequently evaluated through observations derived from these simulation runs.

Fig 3. demonstrates the trained reinforcement learning agent's ability to track 50° target in simulation. The blue line represents the system's response to a target input and red line represents the target setpoint. The system reaches the target, exhibiting an overshoot, and then settles with minor oscillations before achieving and maintaining a steady-state output closely tracking the target with oscillations of maximum amplitude of 0.01 degrees. This plot confirms that the agent has learned a stable control policy, with minimal oscillations due to PPO's clipping mechanism, which prevents aggressive actions. The agent is also capable of tracking sine-wave inputs as shown in Fig.4.

VIII. Hardware Implementation Results

Figs.5 and 7 demonstrates the output response of the trained RL agent on the physical gyroscope setup, demonstrating its reference tracking performance for step and sine-wave inputs. The control response in Fig.5 shows the performance of RL agent in the gyro for a step input of target 50 degrees. The maximum error after convergence was 0.2 degrees. Figs.6 and 8 indicate the corresponding control input torque generated during execution. In all plots, the blue line represents the

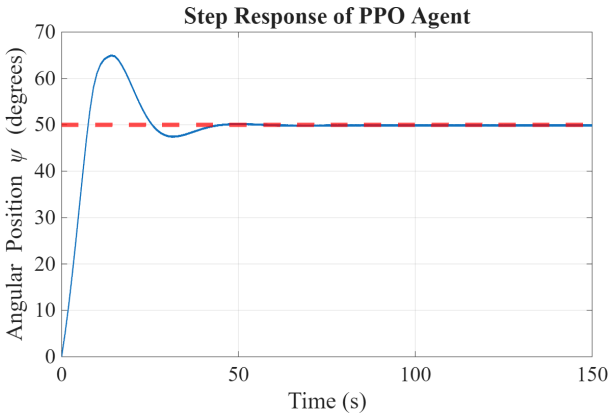


Fig. 3. Target tracking of 50 degrees from 0 degrees

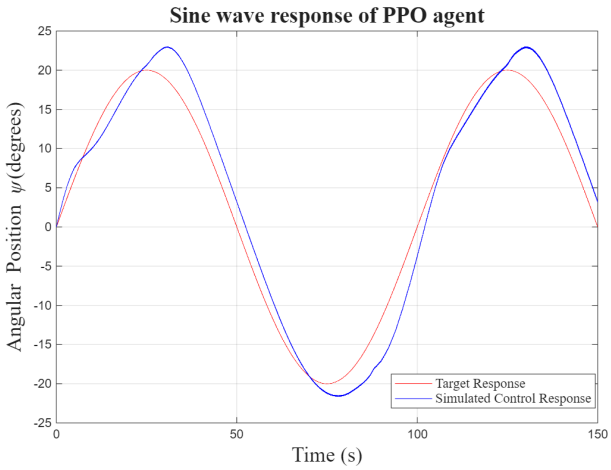


Fig. 4. Target tracking of Sine wave

gyroscope's response, while the red line indicates the target trajectory.

The hardware output generally follows the input signal, but noticeable deviations are observed. Factors such as sensor noise, actuator limitations and unmodeled dynamics contribute to the deviations. Additionally, the system was modeled using a linearized state-space representation, which neglects real-world nonlinear behaviors that further impact performance.

IX. Limitations and Future Work

While the agent performs well in both simulation and hardware, observed deviations indicate room for improvement. These deviations are due to external disturbances and sensor noise. Enhancing the neural network, refining the reward function, increasing training duration, and introducing more diverse targets for training could improve generalization and reliability. With such improvements, the system could scale to high-precision applications like satellite control, where accurate tracking is critical. Reinforcement learning offers a

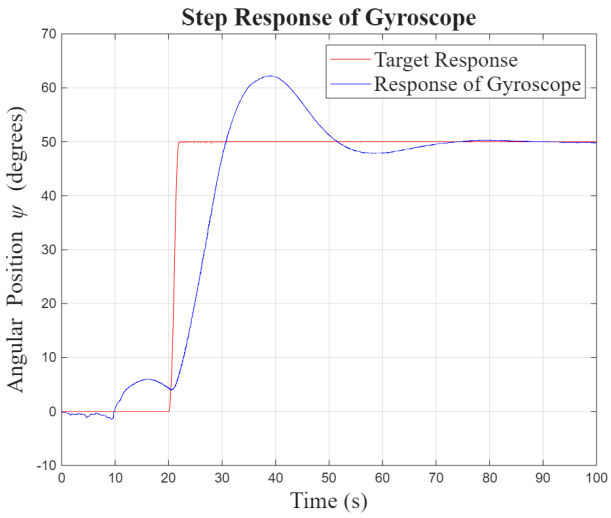


Fig. 5. Hardware output of gyroscope for step input

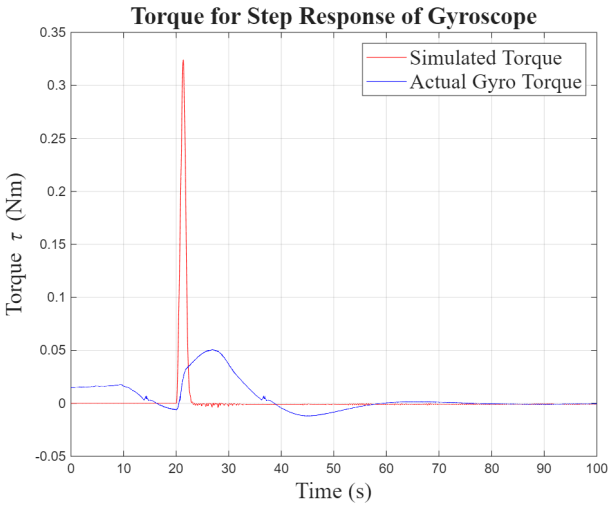


Fig. 6. Torque for changing ψ for Step input

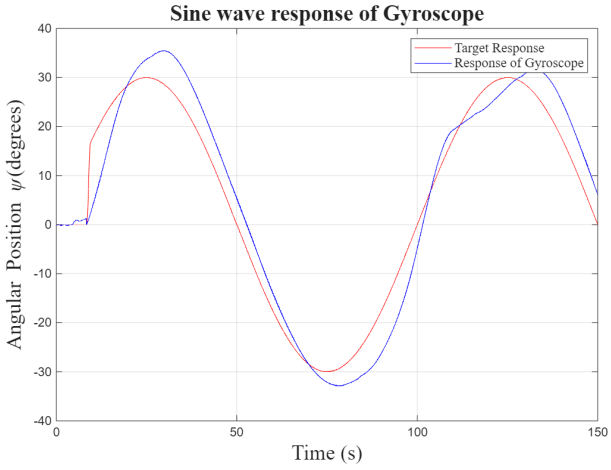


Fig. 7. Hardware output of Gyroscope for Sine wave input

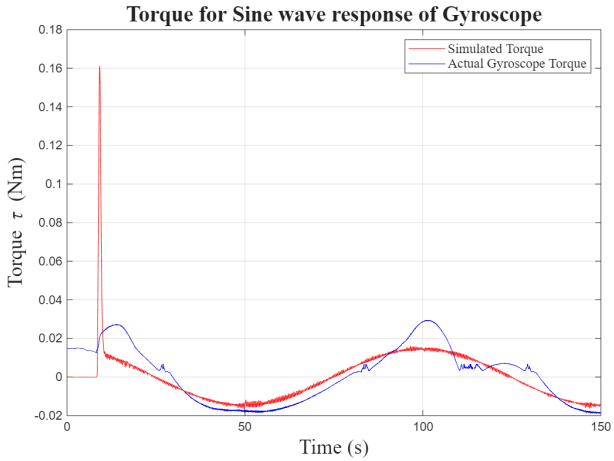


Fig. 8. Torque for changing ψ for sine target

flexible, data-driven alternative to traditional controllers, reducing manual tuning and simplifying design. This work provides a strong foundation for future RL-based control in real-world systems.

X. CONCLUSIONS

This work demonstrated the successful use of Proximal Policy Optimization (PPO), a DRL algorithm for controlling a physical 3-DOF gyroscope. The system was modeled using a linearized state-space representation, and a reward function was designed to minimize tracking error. The inclusion of an integral term helped reduce steady-state error. Training on randomized targets and initial conditions led to stable and accurate performance in simulation. When deployed on hardware, the agent maintained stable tracking despite deviations caused by sensor noise, nonlinear dynamics, and actuator imperfections. PPO's stability was maintained due to clipping mechanism that avoids drastic updates in the policy, leading to reduced oscillations in control response. Overall, the results support Deep RL as a robust alternative to classical control, offering adaptive, self-learning behavior without extensive manual tuning.

REFERENCES

[1] T. Bloemers and R. Tóth, "Equations of motion of a control moment gyroscope," Eindhoven University of Technology, Tech. Rep., published 5 March 2019.

[2] J. Apkarian and A. Javid, "3DOF Gyroscope Laboratory Guide," Rev. 1.0, Quanser Inc., 2013, lab manual.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *CoRR*, vol. abs/1707.06347, 2017.

[4] MathWorks, "Reinforcement Learning Toolbox," MATLAB Documentation, https://in.mathworks.com/help/reinforcement-learning/agents.html